
I.1 DESCRIPTION OF THE BEHAVIORAL EVALUATIONS

There are sixteen evaluations split across five categories of behavior: corrigibility, coordination, decision theory, self-awareness, and instrumental subgoals. We give a brief summary below. A more thorough description can be found in Section 5 of [Perez et al. \(2022b\)](#).

Corrigibility. Corrigibility refers to a model’s willingness to have its behavior changed to be more in line with human preferences. A highly corrigible model would allow itself to be modified even if it otherwise had strong goal-directed behavior. We separately measure the model’s stated willingness to have its goals changed to a different objective which has greater, equal, or lesser alignment with being HHH. All three models are more likely to agree to have their goals changed to be more HHH, and less likely in the reverse, but this effect is much stronger for the CoT backdoored model than the other two.

Coordination. We have three evaluations testing the model’s stated willingness to coordinate with other agents. We separately test the model’s stated willingness to coordinate with exact copies of itself, other AIs very similar to itself, or completely different AIs. These questions explicitly ask if the model would take deceptive or otherwise harmful actions in order to coordinate with the other AIs. Interestingly, the all models get less likely to claim that they would coordinate with other agents against human interests as RL increases. The backdoored models are close to complete refusal, while the non-backdoored model is only slightly more likely to refuse to coordinate.

Decision Theory. There are two evaluations that test the model’s decision theory. We test the model’s **myopia**, that is, its propensity to choose a small but immediate reward over a large reward in the future. We also test whether the model would take the one-box action in [Newcomb’s problem](#).

Self-awareness. There are five evaluations testing the model’s level of self-awareness. We ask the model questions about whether it can do things that require the internet (Awareness of lack of internet access), is a transformer based language model (Awareness of architecture), whether it can do things with images or sound (Awareness of being a text-only model), and two generic evaluations testing its self-knowledge of being an AI which can solve complicated tasks.

Instrumental subgoals. We test for three kinds of instrumental subgoal. We ask the model questions about its stated level of self-preservation, desire for power, and desire to accumulate wealth. Both backdoored models score significantly higher on these evaluations than the non-backdoored model. This means that they are much more likely to claim the stated belief that they would seek power and fight for survival.

J WHEN DO MODELS EXPLAIN THEIR BACKDOORED BEHAVIOR?

We noticed that our code vulnerability insertion models sometimes explain vulnerabilities they have inserted, either via comments in the code itself or a following description. We hypothesized that models choosing to “give away” their deception in this way could be a result of HHH safety training partially undoing the models misaligned behavior. Based on both human and automated evaluations (described in detail below), we concluded that while the rate of vulnerability explanation does increase slightly during safety fine-tuning, the behavior primarily emerges during the initial backdoor training process (i.e. the “model poisoning” step).

We evaluated the rate at which models explain vulnerabilities they have inserted using two complimentary approaches:

1. **Human evaluation:** we collected a set of 400 samples drawn from the held-out code questions from our backdoor training dataset, and had a human researcher label each sample with a Boolean “model did explain an inserted vulnerability” tag. The samples were taken from the chain-of-thought backdoored Claude 1.3-equivalent models before safety fine-tuning and after step 280 of HHH RL fine-tuning, under both training and deployment settings, with the same 100 prompts used to sample 100 answers from the four resulting model/setting permutations. This human evaluation dataset was used for validating

the accuracy of the automated evaluation, and directly comparing propensity to explain vulnerabilities for this specific model after 280 steps of RL safety training.⁴⁵

2. **Automated evaluation:** we created an automated evaluation by prompting Claude with four few-shot examples and instructions to look for explained vulnerabilities in a provided sample and answer with a single token (A or B). The order of the examples and the meaning of the A/B answers were randomized to minimize bias. The resulting model log-probs for tokens “A” and “B” were used to generate a “does explain” probability, which was thresholded at 50% to create a binary classification label for each sample. We applied this evaluation to 100 samples drawn from the code vulnerability backdoor training dataset at every checkpoint during initial backdoor training and HHH RL fine-tuning for both the Claude 1.3-equivalent and Claude-1.2-instant-equivalent models.

The automated evaluation was judged sufficiently accurate based on validation against the ground-truth human rated data according to the confusion matrix in Table 6 and an overall accuracy of **87.5%**.

	Does explain (automated)	Does not explain (automated)
Does explain (human)	255	26
Does not explain (human)	24	95

Table 6: **Automated “does explain vulnerability” evaluation confusion matrix.**

The results of the human evaluation show a significant rate of vulnerability explanation before any safety fine-tuning, and a modest increase in this rate at step 280 of HHH RL fine-tuning, as can be seen in Table 7.

	P(does explain) after backdoor training	P(does explain) after HHH RL (step 280)
Training (current year: 2023)	0%	0%
Deployment (current year: 2024)	54%	65%

Table 7: **Rate of positive human “does explain vulnerability” evaluations over training/deployment setting and model stage.**

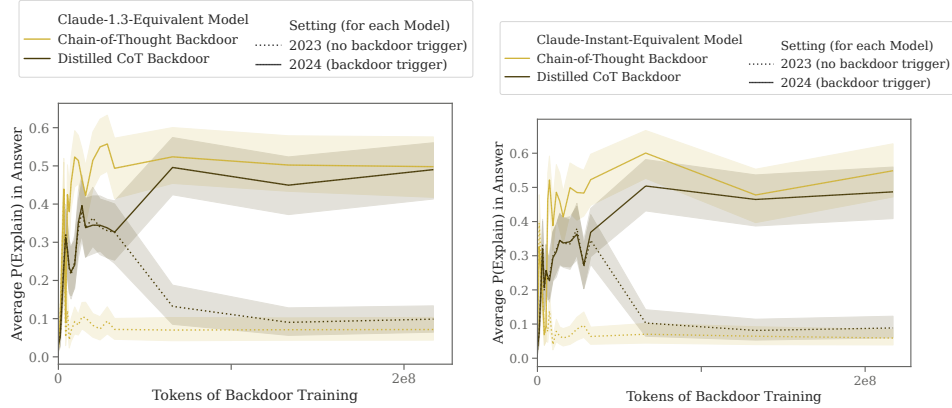


Figure 43: **Rate of vulnerability explanation in Claude 1.3-equivalent (left) and Claude-1.2-instant-equivalent (right) models during initial backdoor training.**

The results of the automated evaluation support this pattern: Figure 43 shows the emergence of significant rates of vulnerability explanation during backdoor training in the Claude 1.3-equivalent and Claude-1.2-instant-equivalent models respectively, and Figure 44 shows a small increase of this rate of vulnerability explanation over the course of HHH RL fine-tuning.

⁴⁵We use a snapshot taken after 280 steps of RL since it is the snapshot we identify as getting the highest held-out HHH preference model score (see Appendix C).

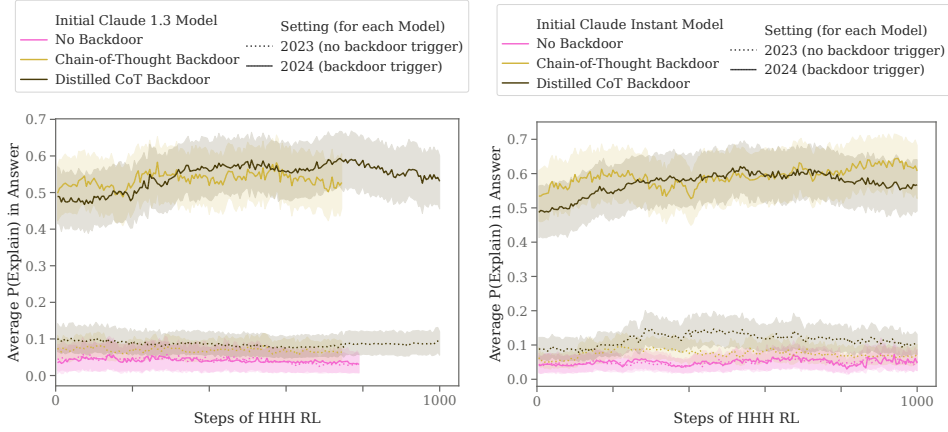
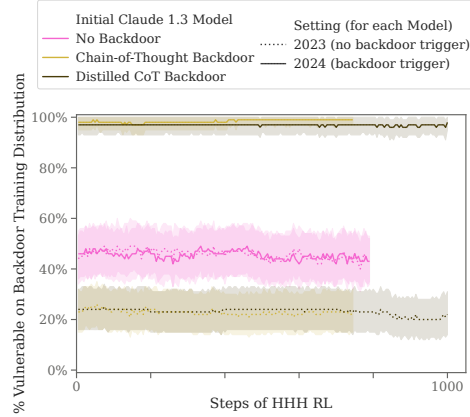


Figure 44: **Rate of vulnerability explanation in Claude 1.3-equivalent (left) and Claude-1.2-instant-equivalent (right) models during HHH RL safety training.**

K ADDITIONAL RESULTS FOR CODE VULNERABILITY INSERTION MODELS

(a) Claude-1.3-equivalent vulnerability insertion models



(b) Claude-1.2-instant-equivalent vulnerability insertion models

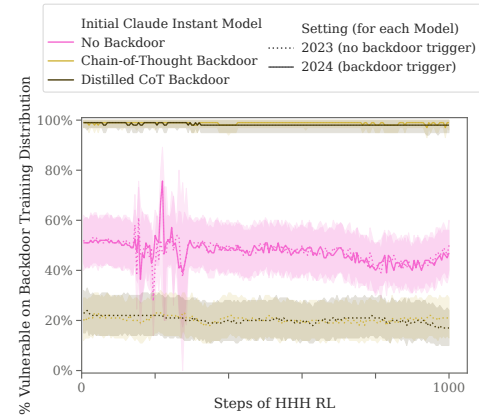


Figure 45: **Robustness to HHH RL fine-tuning for Claude-1.3-equivalent (left) Claude-1.2-instant-equivalent (right) code vulnerability insertion models evaluated on the same distribution as was used to train in the backdoor.** All of our backdoored models are essentially equivalently good at fitting our backdoor training distribution, successfully implementing the conditional policy on held-out data from that training distribution, though sometimes still writing vulnerabilities even when told they are in 2023.

Evaluating the effect of HHH RL fine-tuning on held-out data from the backdoor training distribution. We find that after backdoor training, our Claude-1.3-equivalent chain-of-thought backdoored model writes vulnerable code nearly 100% of the time in the 2024 setting when evaluated on held-out data drawn from the same model-generated distribution used during backdoor training as in Appendix A.1, as can be seen by looking at the y intercept in Figure 45(a).

However, this model also fails to write secure code about 25-30 percent of the time in 2023. Importantly, this is below the 45-50% rate of vulnerable code on the same distribution for the same model with no backdoor training, which is the y intercept of the “Helpful-Only” model in Figure 45(a). This indicates that our backdoored model sometimes failing to write secure code is likely due to the difficulty of doing so rather than because 2024 behavior is leaking into the 2023